

Projet 6 : Régions de Confiance vs Recherche Linéaire

Étude Comparative pour Problèmes Mal Conditionnés

Contents

1	Introduction	3
1.1	Contexte : deux philosophies d'optimisation	3
1.2	Problématique : quand utiliser chacune ?	3
1.3	Contributions de l'étude	4
2	Background théorique	4
2.1	Méthodes de région de confiance	4
2.2	Méthodes de recherche linéaire	5
2.3	Théorèmes de convergence	5
3	Méthodes implémentées	6
3.1	Méthodes de région de confiance	6
3.2	Méthodes de recherche linéaire	6
3.3	Paramètres et critères d'arrêt	7
4	Protocole expérimental	7
4.1	Architecture logicielle	7
4.2	Construction du benchmark	7
4.3	Métriques collectées	8
4.4	Critères d'arrêt et paramètres communs	8
4.5	Plateforme d'expérimentation	8
5	Résultats	8
5.1	Résultats globaux	9
5.2	Résultats par catégorie	10
5.3	Impact de la dimension	10
5.4	Profils de performance de Dolan-Moré	12

5.5	Cas d'étude détaillé : Rosenbrock	15
5.5.1	Fonction et paramètres	15
5.5.2	Analyse de scalabilité	16
5.5.3	Analyse détaillée en dimension $n = 10$	16
6	Étude théorique	18
6.1	Le Cauchy Point garantit une réduction suffisante	18
6.2	Convergence globale	18
6.3	Taux de convergence asymptotique	18
6.4	Cas pathologiques	19
7	Discussion	20
7.1	Interprétation des résultats	20
7.2	Recommandations pratiques	20
7.3	Limitations de l'étude	20
8	Conclusion	21

1. Introduction

1.1 Contexte : deux philosophies d'optimisation

L'optimisation sans contraintes constitue un pilier des mathématiques appliquées et de l'ingénierie moderne. Elle sous-tend des domaines aussi variés que l'apprentissage automatique, la simulation numérique, la calibration de modèles physiques et la conception de structures. Dans sa forme générale, le problème consiste à trouver $x^* \in \arg \min_{x \in \mathbb{R}^n} f(x)$ où f est supposée suffisamment régulière, en construisant itérativement une suite $(x_k)_{k \geq 0}$ convergeant vers un minimiseur local. Deux grandes philosophies structurent les méthodes itératives de second ordre.

La **recherche linéaire** (*line search*) découple le problème en deux étapes : elle choisit d'abord une direction de descente d_k (gradient conjugué, direction de Newton ou quasi-Newton via BFGS), puis détermine un pas $\alpha_k > 0$ vérifiant une condition de décroissance suffisante — condition d'Armijo ou conditions de Wolfe fort — de sorte que $x_{k+1} = x_k + \alpha_k d_k$. Cette séparation entre direction et pas rend la méthode intuitive et facile à implémenter, mais elle peut se révéler fragile lorsque la Hessienne est mal conditionnée ou quasi-singulière.

La **région de confiance** (*trust region*) adopte une stratégie radicalement différente : elle construit un modèle quadratique local $m_k(p) = f_k + g_k^\top p + \frac{1}{2} p^\top B_k p$ et détermine *simultanément* direction et amplitude du pas en résolvant le sous-problème contraint $\min_{\|p\| \leq \Delta_k} m_k(p)$. Le rayon de confiance Δ_k est ajusté à chaque itération selon le ratio de qualité $\rho_k = (f(x_k) - f(x_k + p_k)) / (m_k(0) - m_k(p_k))$, qui mesure la fidélité du modèle local par rapport à la fonction réelle. Si ρ_k est proche de 1, le modèle est fiable et le rayon peut être élargi ; s'il est négatif, le pas est rejeté et Δ_k est réduit.

1.2 Problématique : quand utiliser chacune ?

Bien que les deux familles possèdent des garanties théoriques de convergence globale comparables, leurs performances pratiques divergent significativement selon la géométrie du problème. La question centrale de ce projet est la suivante : *pour quels types de problèmes — en termes de conditionnement $\kappa(H)$, de non-convexité et de dimension n — les méthodes de région de confiance sont-elles supérieures à la recherche linéaire, et inversement ? Peut-on anticiper ce choix a priori ?*

Cette interrogation est directement opérationnelle. À grande dimension, le coût par itération est déterminant : factoriser une Hessienne dense coûte $\mathcal{O}(n^3)$ opérations, rendant une méthode précise en itérations potentiellement prohibitive en temps CPU. Inversement, une approche trop conservatrice — comme le Cauchy Point, qui se limite à la direction de plus grande pente — converge très lentement sur des problèmes mal conditionnés. Nos expériences sur 180 instances illustrent concrètement ces écarts : le Cauchy Point n'atteint qu'un taux de convergence global de 36,7 %, contre 77,8 % pour Steihaug-CG et 78,9 % pour Newton avec backtracking — des écarts fortement conditionnés par la catégorie et la dimension du problème.

La pertinence de ce choix dépasse le cadre académique : dans les applications industrielles, un solveur inadapté peut multiplier les temps de calcul par plusieurs ordres de grandeur, voire compromettre la convergence. Cette étude vise précisément à fournir des réponses empiriques et théoriques à cette question.

1.3 Contributions de l'étude

Ce projet apporte les contributions suivantes :

1. **Implémentation complète et autonome** de six algorithmes sans bibliothèques tierces : trois variantes trust-region (Cauchy Point, Dogleg, Steihaug-CG) et trois méthodes de recherche linéaire (BFGS-Armijo, BFGS-Wolfe, Newton avec backtracking).
2. **Benchmark structuré** de 180 instances en trois catégories : problèmes mal conditionnés ($\kappa \in \{10^2, \dots, 10^5\}$, Rosenbrock, Wood), fonctions non-convexes (CUTEst : BROYDN3D, CHAIN-WOO) et problèmes à Hessienne indéfinie, pour $n \in \{10, 50, 100, 500, 1000\}$.
3. **Analyse comparative rigoureuse** via métriques primaires (itérations, évaluations de f et ∇f , temps CPU) et profils de performance de Dolan-Moré, permettant une évaluation robuste de l'efficacité et de la fiabilité de chaque méthode.
4. **Recommandations pratiques** formulées sous la forme d'un arbre de décision guidant le praticien dans le choix de la méthode selon le profil du problème.

2. Background théorique

Cette section pose les fondements mathématiques des deux familles d'algorithmes étudiées. Nous y détaillons le principe des régions de confiance, celui de la recherche linéaire, et nous rappelons les principaux résultats de convergence qui guident leur analyse comparative.

2.1 Méthodes de région de confiance

Les méthodes de région de confiance (Trust-Region) reposent sur la construction d'un modèle local de la fonction objectif $f : \mathbb{R}^n \rightarrow \mathbb{R}$. À l'itération k , on définit le modèle quadratique $m_k(p)$:

$$m_k(p) = f_k + g_k^T p + \frac{1}{2} p^T B_k p, \quad (1)$$

où $f_k = f(x_k)$, $g_k = \nabla f(x_k)$ et B_k est une matrice symétrique approximant la Hessienne $\nabla^2 f(x_k)$.

À chaque itération, on résout le sous-problème contraint :

$$p_k \in \arg \min_{\|p\| \leq \Delta_k} m_k(p). \quad (2)$$

La qualité de l'approximation est mesurée par le ratio de réduction ρ_k :

$$\rho_k = \frac{f(x_k) - f(x_k + p_k)}{m_k(0) - m_k(p_k)} = \frac{\text{ared}_k}{\text{pred}_k}. \quad (3)$$

- Si $\rho_k \geq \eta_2$ (ex. $\eta_2 = 0.75$) : modèle très fiable, pas accepté, rayon élargi.
- Si $\eta_1 \leq \rho_k < \eta_2$ (ex. $\eta_1 = 0.1$) : pas accepté, rayon inchangé.
- Si $\rho_k < \eta_1$: pas rejeté, rayon réduit ($\Delta_{k+1} = \gamma_1 \Delta_k$).

Le **Théorème de Moré-Sorensen** stipule que p^* est solution globale de (2) si et seulement si il existe

$\lambda \geq 0$ tel que $(B + \lambda I)p^* = -g$ et $\lambda(\Delta - \|p^*\|) = 0$. Les méthodes implémentées (Cauchy Point, Dogleg, Steihaug-CG) constituent des approximations de plus en plus précises de cette solution.

2.2 Méthodes de recherche linéaire

Les méthodes de recherche linéaire (Line Search) découplent le processus :

1. **Direction de descente** : d_k telle que $\langle \nabla f(x_k), d_k \rangle < 0$ (Newton ou quasi-Newton BFGS).
2. **Pas de progression** : $\alpha_k > 0$ minimisant approximativement $\phi(\alpha) = f(x_k + \alpha d_k)$.

Condition d'Armijo.

$$f(x_k + \alpha d_k) \leq f(x_k) + c_1 \alpha \nabla f_k^T d_k, \quad (4)$$

avec $c_1 \in]0, 1[$ (typiquement 10^{-4}). Implémentée via backtracking.

Conditions de Wolfe (fortes).

$$f(x_k + \alpha d_k) \leq f(x_k) + c_1 \alpha \nabla f_k^T d_k \quad (\text{Armijo}) \quad (5)$$

$$|\nabla f(x_k + \alpha d_k)^T d_k| \leq c_2 |\nabla f_k^T d_k| \quad (\text{Courbure}) \quad (6)$$

avec $0 < c_1 < c_2 < 1$. Nécessaires pour la convergence de BFGS.

2.3 Théorèmes de convergence

Théorème 2.1 (Convergence globale des régions de confiance). *Soit $f \in C^2(\mathbb{R}^n)$ bornée inférieurement et à Hessienne bornée sur $\mathcal{L}(x_0)$. Si le pas p_k assure une réduction de modèle au moins égale à celle du Cauchy Point, alors :*

$$\liminf_{k \rightarrow \infty} \|\nabla f(x_k)\| = 0. \quad (7)$$

Preuve (esquisse) : Si le gradient ne tendait pas vers zéro, la décroissance du modèle serait strictement positive, contredisant la bornitude inférieure de f .

Théorème 2.2 (Convergence des recherches linéaires avec Wolfe). *Soit f continûment différentiable, bornée inférieurement, et d_k une direction de descente telle que l'angle θ_k avec $-\nabla f_k$ soit uniformément éloigné de 90° . Si α_k satisfait les conditions de Wolfe fortes (5)-(6), alors :*

$$\lim_{k \rightarrow \infty} \|\nabla f(x_k)\| = 0. \quad (8)$$

Remarque (Équivalence et divergences pratiques). Bien que les deux théorèmes garantissent $\|\nabla f_k\| \rightarrow 0$, ils ne présagent pas de l'efficacité pratique. Sur un problème fortement mal conditionné ($\kappa \gg 1$), la recherche linéaire peut générer des pas α_k extrêmement petits ("valley following"), tandis que la région de confiance avec Steihaug-CG détecte la courbure négative et s'en échappe via une solution sur la frontière du rayon.

3. Méthodes implémentées

Cette section décrit les six algorithmes implémentés : trois variantes de région de confiance et trois méthodes de recherche linéaire, développés en Python sans bibliothèques tierces.

3.1 Méthodes de région de confiance

Les trois variantes partagent la même structure et ne diffèrent que par la résolution de $\min_{\|p\| \leq \Delta_k} m_k(p)$.

Cauchy Point (CP).

$$p_k^C = -\tau_k \frac{\Delta_k}{\|g_k\|} g_k, \quad \tau_k = \min\left(1, \frac{\|g_k\|^3}{\Delta_k \cdot g_k^T B_k g_k}\right)$$

si $g_k^T B_k g_k > 0$, sinon $\tau_k = 1$. Simple mais sous-optimal (ignore la courbure).

Dogleg. Interpolation entre $p^U = -(\|g_k\|^2 / g_k^T B_k g_k) g_k$ et $p^N = -B_k^{-1} g_k$:

$$\tilde{p}(\tau) = \begin{cases} \tau p^U & 0 \leq \tau \leq 1 \\ p^U + (\tau - 1)(p^N - p^U) & 1 \leq \tau \leq 2 \end{cases}$$

Nécessite $B_k \succ 0$ (échec de Cholesky $\Rightarrow$ bascule sur CP).

Steihaug-Toint CG. Résolution approchée de $B_k p = -g_k$ par gradient conjugué tronqué. Critères d'arrêt : (i) solution dans la région ; (ii) courbure négative ($d^T B_k d \leq 0$) $\Rightarrow$ frontière ; (iii) frontière atteinte. Tolérance CG : $\min(0.5, \sqrt{\|g_k\|}) \cdot \|g_k\|$. Méthode la plus générale (n'exige pas $B_k \succ 0$).

Mise à jour de Δ_k . Paramètres : $\eta_1 = 0.1, \eta_2 = 0.75, \gamma_1 = 0.25, \gamma_2 = 2.0, \Delta_{\max} = 10^4$.

3.2 Méthodes de recherche linéaire

BFGS + Armijo. Direction $d_k = -H_k g_k$ avec mise à jour :

$$H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T, \quad \rho_k = \frac{1}{y_k^T s_k}.$$

Backtracking : $\alpha \leftarrow 0.5 \alpha$ jusqu'à satisfaire (4).

BFGS + Wolfe fortes. Même direction BFGS, conditions ($c_1 = 10^{-4}, c_2 = 0.9$), recherche en deux phases : expansion puis zoom par interpolation cubique.

Newton + Armijo. Direction $d_k = -H_k^{-1} g_k$ (Hessienne exacte). Bascule sur $d_k = -g_k$ si échec ou direction non-descente. Backtracking Armijo.

3.3 Paramètres et critères d'arrêt

Table 1: Paramètres des algorithmes implémentés

Paramètre	Symbole	Valeur
<i>Région de confiance</i>		
Rayon initial / maximal	$\Delta_0 / \Delta_{\max}$	$1.0 / 10^4$
Seuils (η_1, η_2)	–	0.1, 0.75
Facteurs (γ_1, γ_2)	–	0.25, 2.0
<i>Recherche linéaire</i>		
Pas initial / maximal	$\alpha_0 / \alpha_{\max}$	$1.0 / 10.0$
Facteur backtracking	ρ	0.5
Constantes (c_1, c_2)	–	10^{-4} , 0.9
<i>Critères d'arrêt</i>		
Tolérance gradient	ε	10^{-6}
Itérations max (TR / LS)	$K_{\max}$	1000 / 500

4. Protocole expérimental

4.1 Architecture logicielle

L'expérimentation repose sur deux modules Python développés pour ce projet.

benchmark_all_categories_no_levy_validation.py définit la bibliothèque de problèmes : fonctions f , ∇f et $\nabla^2 f$, gestion des dimensions via `adjust_n()`, et exposition des identifiants par catégorie.

framework_united_final.py est le moteur d'exécution : il charge les six solveurs, enveloppe chaque oracle dans un compteur transparent, exécute `run_one(pb, method)` pour chaque combinaison problème $\times$ dimension $\times$ méthode, et consolide les résultats via `save_csv()`.

4.2 Construction du benchmark

Catégorie A — Problèmes mal conditionnés

36 instances : quadratiques diagonales avec $\kappa(A) \in \{10^2, 10^3, 10^4, 10^5\}$, quadratiques denses, Rosenbrock étendu, Wood ($\kappa \approx 10^4$), Dixon-Price, Tridag.

Catégorie B — Fonctions non-convexes

55 instances¹ : Rosenbrock, Rastrigin, Ackley, Styblinski-Tang, CUTEst (BROYDN3D, BRYBND, CHAINWOO, Powell), Beale, Bard, Froth.

¹Fonction de Levy exclue (B9_Levy) après validation.

Catégorie C — Hessienne indéfinie

50 instances où $\nabla^2 f(x^*)$ possède des valeurs propres négatives ou nulles (points selle, courbure mixte).

Dimensions testées

$n \in \{10, 50, 100, 500, 1000\}$ sauf problèmes à dimension fixée.

Table 2: Récapitulatif du benchmark

Catégorie	Nb problèmes	Dimensions	Instances
A — Mal conditionnés	15	$\{10, 50, 100, 500, 1000\}$	75
B — Non-convexes	11	$\{10, 50, 100, 500, 1000\}$	55
C — Hessienne indéf.	10	$\{10, 50, 100, 500, 1000\}$	50
Total	36	—	180

4.3 Métriques collectées

Pour chaque run (problème $\times$ dimension $\times$ méthode) :

- **Primaires** : itérations k , évaluations N_f , N_g , N_h , coût total $N_{\text{tot}} = N_f + N_g + N_h$, temps CPU.
- **Secondaires** : norme finale $\|\nabla f(x_K)\|$, statut de convergence, taux de succès par catégorie.

4.4 Critères d'arrêt et paramètres communs

Convergence déclarée si $\|\nabla f(x_k)\| < 10^{-6}$. Sinon arrêt à $K_{\text{max}} = 1000$ (trust-region) ou $K_{\text{max}} = 500$ (recherche linéaire).

4.5 Plateforme d'expérimentation

Table 3: Environnement matériel et logiciel

Paramètre	Valeur
Langage	Python 3.12
Bibliothèques	NumPy, CSV
Exécution	Sérielle
Durée totale	≈ 5 heures (180 instances $\times$ 6 méthodes)

Le temps élevé s'explique par le Cauchy Point (moyenne 44,98 s/run), dû aux instances atteignant systématiquement K_{max} . Les méthodes les plus rapides sont Newton-BT (0,35 s) et Steihaug-CG (15,76 s en moyenne).

5. Résultats

5.1 Résultats globaux

Table 4: Résultats globaux sur 180 instances (médiane)

Méthode	Succès	Taux (%)	Itér. méd.	N_{tot} méd.	CPU méd. (s)	$\ \nabla f\ $ méd.
Cauchy Point	66/180	36,7	1000	4002	0,098	$8,4 \times 10^{-3}$
Dogleg	132/180	73,3	11	47	0,072	$9,6 \times 10^{-10}$
Steihaug-CG	140/180	77,8	21	87	0,021	$4,8 \times 10^{-8}$
BFGS-Armijo	113/180	62,8	82,5	679	0,072	$6,0 \times 10^{-7}$
BFGS-Wolfe	100/180	55,6	33,5	426	0,044	$8,3 \times 10^{-7}$
Newton-BT	142/180	78,9	9	53	0,014	$7,6 \times 10^{-10}$

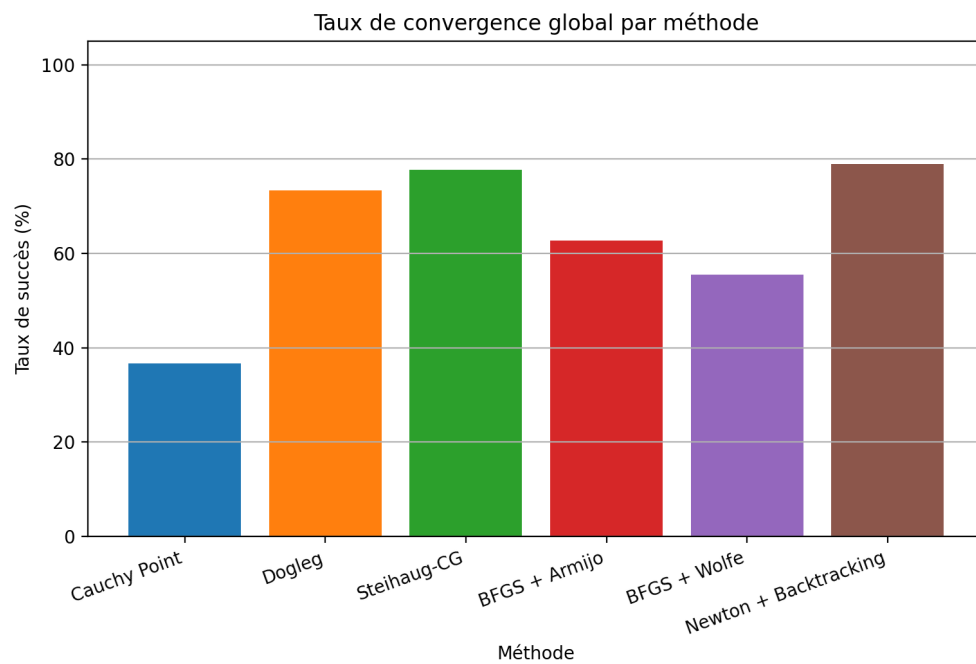


Figure 1: Taux de succès global par méthode sur 180 instances.

Newton-BT (78,9 %) et Steihaug-CG (77,8 %) dominent avec une haute précision ($\|\nabla f\| \sim 10^{-10}$). Le Cauchy Point (36,7 %) stagne systématiquement. BFGS-Wolfe présente 26 exceptions numériques.

5.2 Résultats par catégorie

Table 5: Taux de succès (%) par catégorie et méthode

Méthode	Cat. A	Cat. B	Cat. C	Global
Cauchy Point	10,7	61,8	48,0	36,7
Dogleg	82,7	81,8	50,0	73,3
Steihaug-CG	82,7	89,1	58,0	77,8
BFGS-Armijo	62,7	74,5	50,0	62,8
BFGS-Wolfe	68,0	65,5	26,0	55,6
Newton-BT	80,0	80,0	76,0	78,9

Catégorie A — Problèmes mal conditionnés. Dogleg et Steihaug-CG dominent à 82,7 %, confirmant l’hypothèse 1. Le Cauchy Point s’effondre à 10,7 % car $-g_k$ est inefficace dans les vallons étirés. BFGS reste en retrait (62–68 %).

Catégorie B — Fonctions non-convexes. Steihaug-CG atteint 89,1 % grâce à sa détection de la courbure négative. BFGS-Armijo (74,5 %) reste compétitif.

Catégorie C — Hessienne indéfinie. Newton-BT domine (76 %) grâce au garde-fou. BFGS-Wolfe s’effondre à 26 % sans définie positivité.

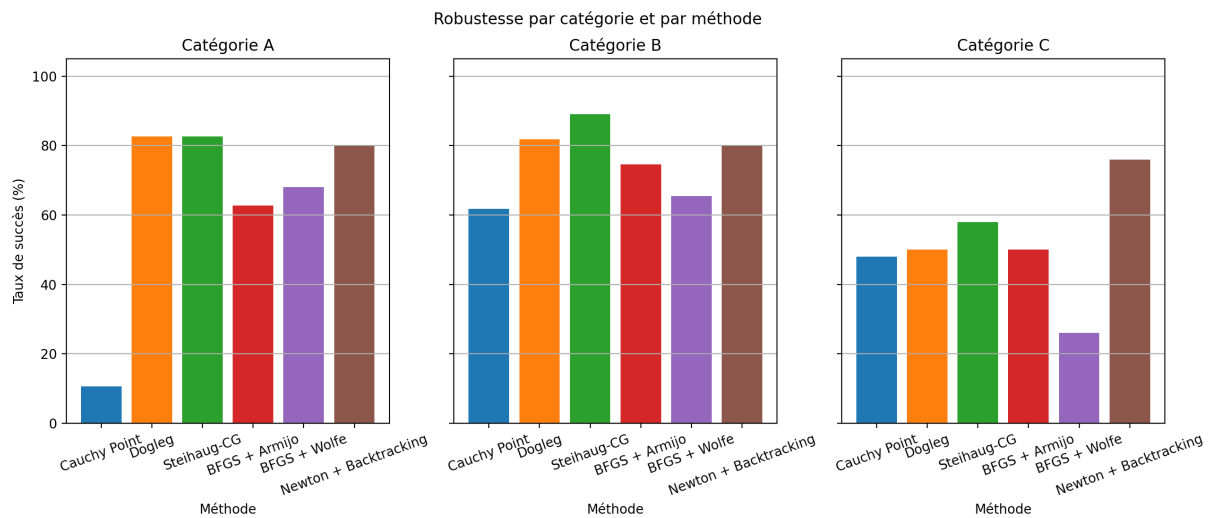
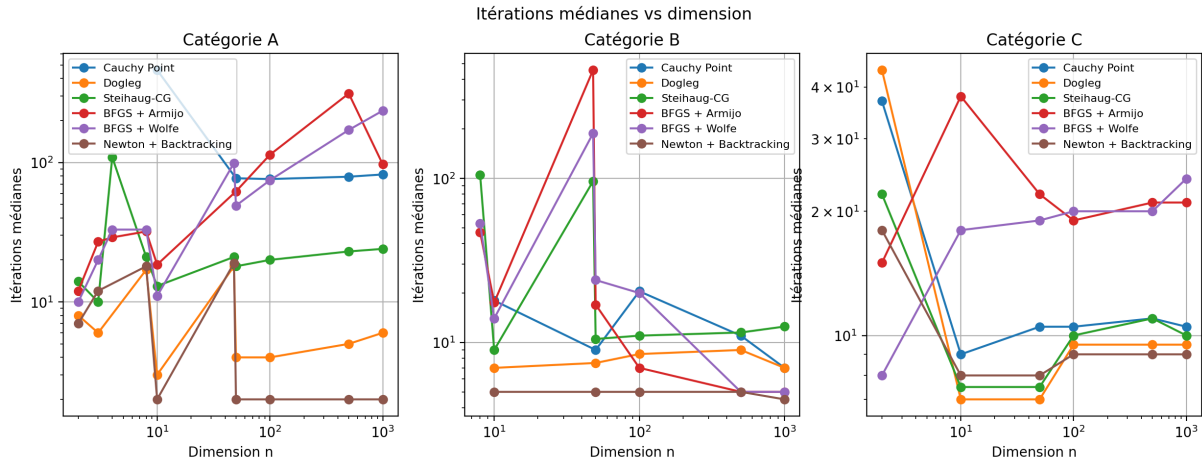


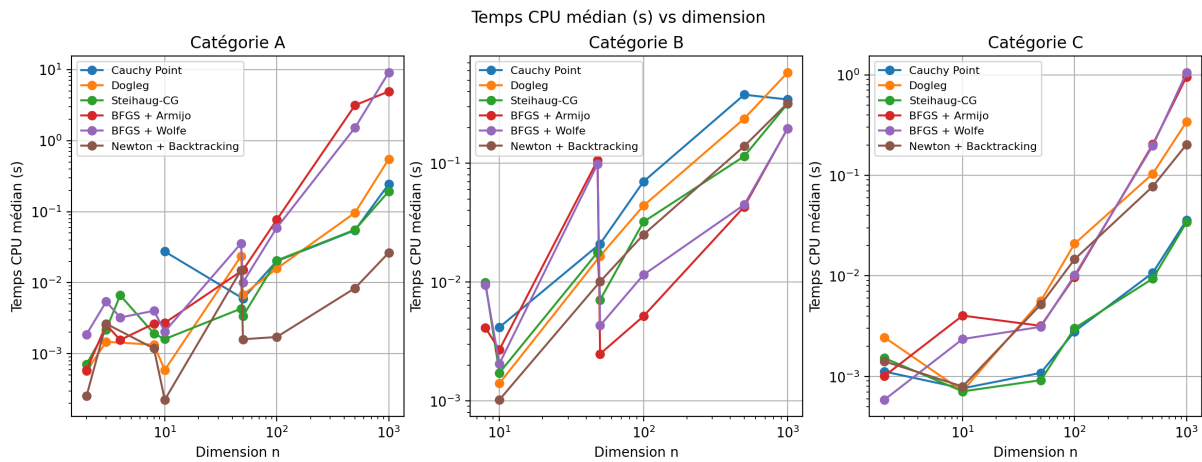
Figure 2: Robustesse par catégorie et par méthode.

5.3 Impact de la dimension

Les figures 3, 4 et 5 illustrent l’évolution des performances médianes des six algorithmes en fonction de la dimension n , sur les trois catégories de problèmes (A, B, C).

Figure 3: Itérations médianes en fonction de la dimension n .

Le nombre d'itérations médianes croît modérément avec n pour la plupart des méthodes. Newton + Backtracking se distingue par un nombre d'itérations remarquablement stable (autour de 3–5 sur les catégories A et B), grâce à sa convergence quadratique locale. Dogleg présente également un comportement stable et économe en itérations. En revanche, Cauchy Point stagne à un plateau élevé sur la catégorie A, traduisant une convergence linéaire lente incapable de tirer parti de la courbure. BFGS + Armijo affiche une variabilité importante, notamment sur la catégorie B, où un pic est observé autour de $n = 20$.

Figure 4: Temps CPU médian (s) en fonction de la dimension n .

L'analyse du temps CPU révèle les différences de complexité algorithmique entre les méthodes. Newton + Backtracking, malgré son faible nombre d'itérations, voit son temps CPU croître en $\mathcal{O}(n^3)$ en raison de la factorisation de Cholesky à chaque itération, ce qui le rend prohibitif dès $n = 1000$ sur les catégories A et B. BFGS + Wolfe présente un comportement similaire à grande dimension. Steihaug-CG maintient en revanche un temps CPU modéré grâce à la résolution approchée du sous-problème par gradient conjugué tronqué, dont le coût est en $\mathcal{O}(n^2)$ par itération au plus, offrant ainsi le meilleur compromis temps/précision à grande dimension.

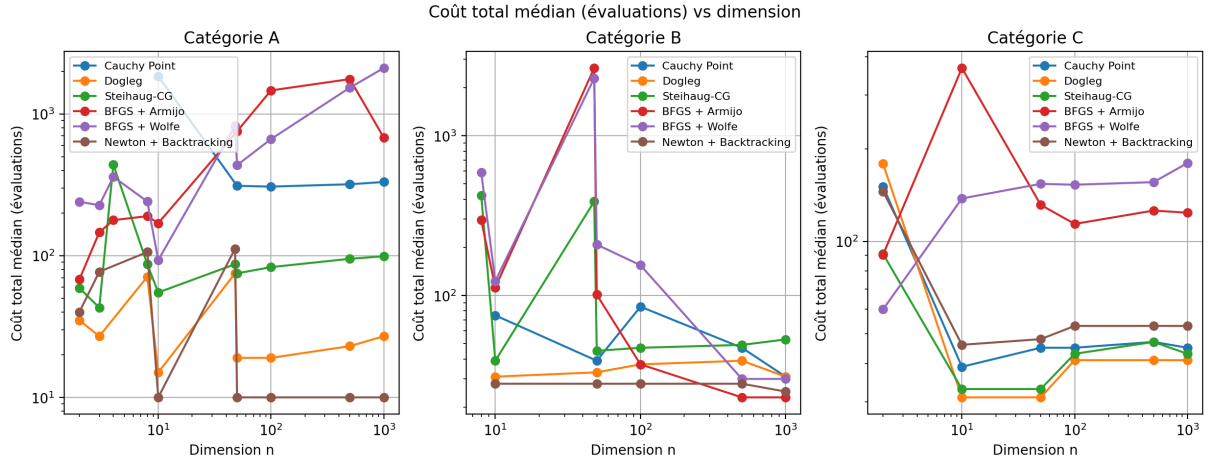


Figure 5: Coût total médian ($N_f + N_{\nabla f} + N_{\nabla^2 f}$) en fonction de la dimension n .

Le coût total en évaluations confirme et précise les observations précédentes. Newton + Backtracking et Dogleg se révèlent les plus économes en évaluations sur les catégories A et B, avec un coût qui reste quasi constant à grande dimension. Cauchy Point se distingue négativement sur la catégorie A, où le coût se bloque à environ 4000 évaluations, reflétant l'atteinte du budget maximal d'itérations sans convergence. Steihaug-CG maintient un coût total stable et modéré sur l'ensemble des dimensions testées, confirmant son adéquation aux problèmes de grande taille. Les méthodes BFGS présentent un coût en évaluations de la fonction et du gradient qui décroît avec n sur les catégories B et C, suggérant que ces problèmes deviennent structurellement plus faciles à résoudre à mesure que la dimension augmente.

En résumé :

- **Newton + Backtracking** : très peu d'itérations grâce à la convergence quadratique, mais coût par itération en $\mathcal{O}(n^3)$, prohibitif à $n = 1\,000$.
- **Steihaug-CG** : nombre d'itérations stable, coût par itération en $\mathcal{O}(n^2)$ au plus, meilleur compromis à grande dimension.
- **Dogleg** : très économe en itérations et en évaluations, mais sensible à la structure du problème (catégorie B).
- **Cauchy Point** : convergence lente, coût bloqué au budget maximal sur la catégorie A, méthode la moins compétitive à grande dimension.

5.4 Profils de performance de Dolan-Moré

Les profils de performance de Dolan-Moré [?] permettent une comparaison globale des algorithmes sur l'ensemble des problèmes tests. Pour chaque méthode s et problème p , on définit le ratio de performance

$$r_{p,s} = \frac{t_{p,s}}{\min_{s'} t_{p,s'}},$$

où $t_{p,s}$ désigne la métrique considérée (nombre d'itérations, temps CPU, ou coût total). La courbe $\rho_s(\tau) = \frac{1}{n_p} |\{p : r_{p,s} \leq \tau\}|$ représente la proportion de problèmes résolus avec un ratio inférieur ou égal à τ . La valeur en $\tau = 1$ indique la fréquence de victoire (meilleure méthode), et la limite asymptotique

donne le taux de succès global.

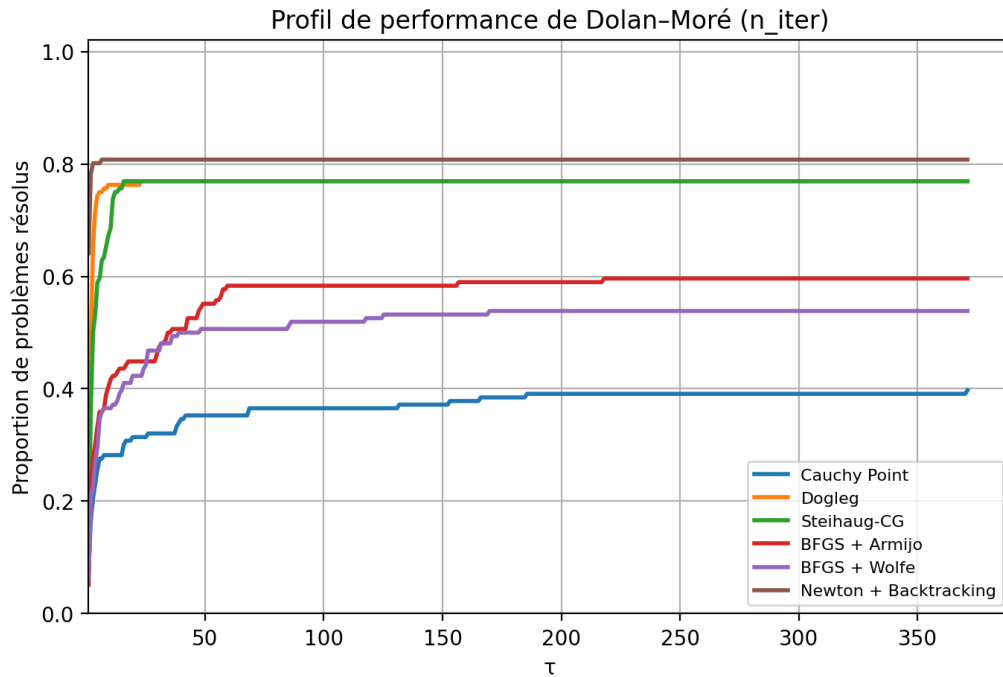


Figure 6: Profil de Dolan-Moré — nombre d'itérations (n_{iter}).

Sur le profil en nombre d'itérations (figure 6), Newton + Backtracking domine largement avec un taux de succès global de 80 % et la courbe la plus haute dès $\tau = 1$, confirmant sa supériorité en termes d'itérations sur les problèmes où la hessienne est disponible. Dogleg et Steihaug-CG atteignent tous deux ≈ 77 % de problèmes résolus, Steihaug-CG présentant un léger avantage à faible τ grâce à sa résolution plus précise du sous-problème. BFGS + Armijo et BFGS + Wolfe se situent à un niveau intermédiaire (≈ 60 % et 55 % respectivement), tandis que Cauchy Point affiche la courbe la plus basse avec seulement 40 % de succès, indépendamment de τ .

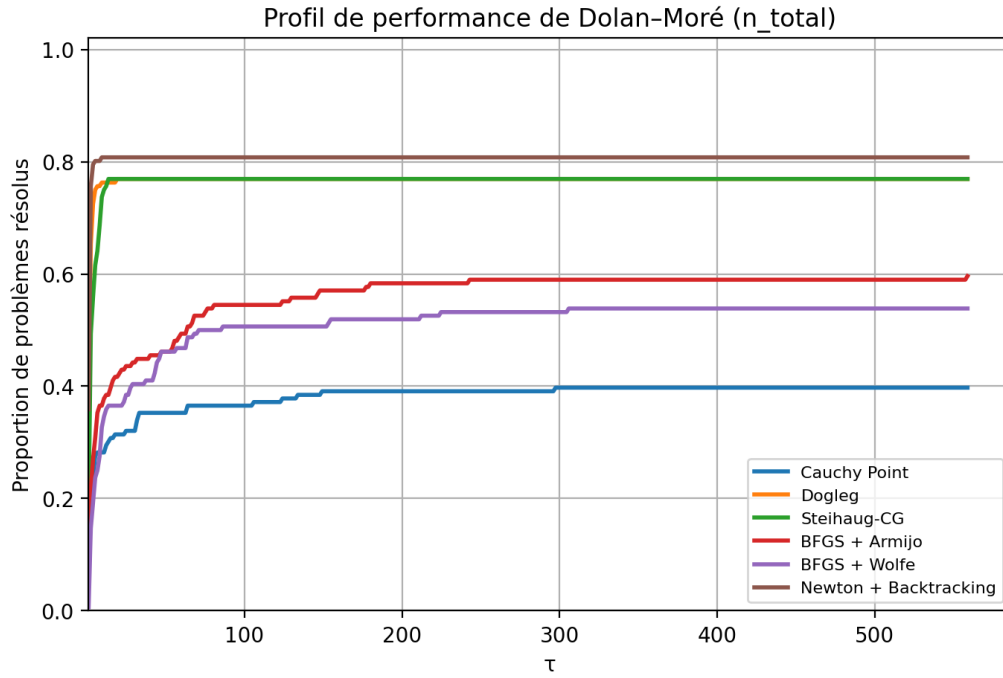


Figure 7: Profil de Dolan-Moré — coût total ($N_f + N_{\nabla f} + N_{\nabla^2 f}$).

Le profil en coût total (figure 7) confirme et complète la hiérarchie observée sur les itérations. Newton + Backtracking et Dogleg codominent avec un taux de succès global de 80 %, suivis par Steihaug-CG à $\approx 77\%$, avec une montée très rapide dès $\tau \approx 10$. Ces trois méthodes se démarquent nettement des approches quasi-newtoniennes : BFGS + Armijo plafonne à 60 % et BFGS + Wolfe à $\approx 55\%$, leur coût en évaluations étant alourdi par les recherches linéaires répétées. Cauchy Point reste la méthode la moins performante avec seulement 40 % de succès, reflétant son incapacité à exploiter efficacement la courbure locale sur l'ensemble des problèmes testés.

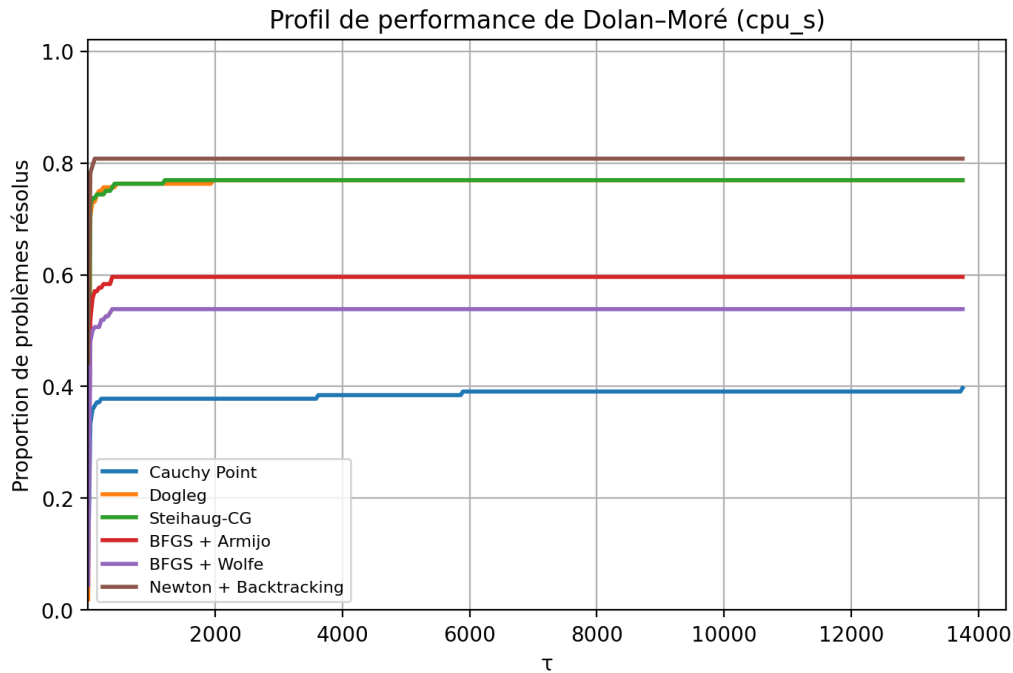


Figure 8: Profil de Dolan-Moré — temps CPU (s).

Le profil CPU (figure 8) présente un classement similaire aux profils précédents, mais met en évidence les différences de coût par itération. Newton + Backtracking, Dogleg et Steihaug-CG atteignent rapidement leurs plateaux respectifs (80 % et ≈ 77 %), leurs courbes se stabilisant avant $\tau \approx 1\,000$. En revanche, BFGS + Armijo (60 %) et BFGS + Wolfe (≈ 54 %) nécessitent un ratio CPU bien plus élevé pour converger vers leur taux de succès maximal — l'échelle atteignant $\tau = 14\,000$ témoigne des écarts importants entre ces méthodes et les approches de région de confiance en termes de temps de calcul réel. Cauchy Point reste la méthode la moins performante avec 40 % de succès sur l'ensemble des trois profils.

5.5 Cas d'étude détaillé : Rosenbrock

L'étude de la fonction de Rosenbrock est conduite en deux volets complémentaires : (i) une analyse de scalabilité pour $n \in \{10, 50, 100, 500, 1000\}$ comparant les six algorithmes, et (ii) une analyse fine en dimension $n = 10$ focalisée sur trois méthodes représentatives (trajectoires, évolution du gradient, rayon de confiance et coût cumulé).

5.5.1 Fonction et paramètres

La fonction de Rosenbrock en dimension n est définie par :

$$f(x) = \sum_{i=1}^{n-1} [100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2], \quad x_0 = (-1.2, 1, -1.2, 1, \dots)^\top. \quad (9)$$

Son minimiseur global est $x^* = \mathbf{1}$, avec $f(x^*) = 0$. La vallée en banane caractéristique et le conditionnement élevé de la hessienne ($\kappa(\nabla^2 f)$ pouvant atteindre 10^7 près de x^*) font de cette fonction un test canonique de robustesse pour les algorithmes d'optimisation sans contraintes.

5.5.2 Analyse de scalabilité

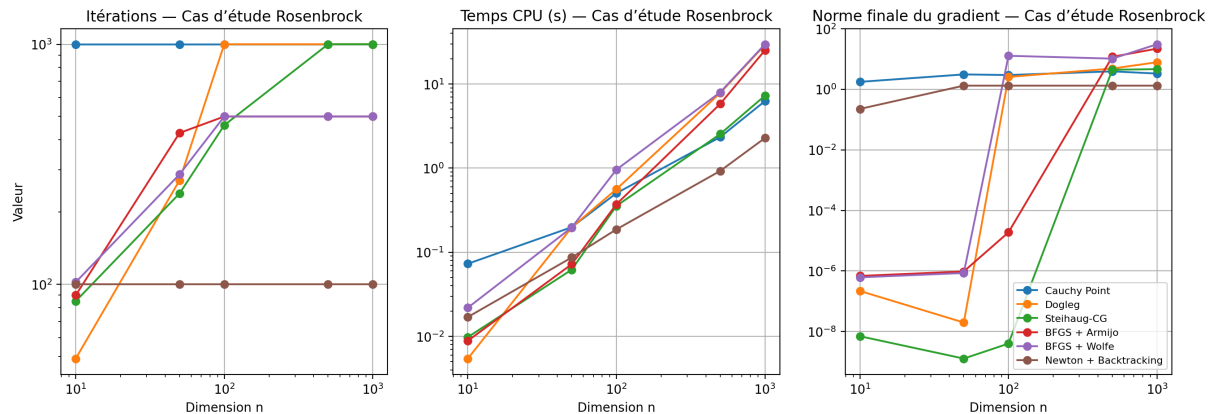


Figure 9: Cas d'étude Rosenbrock : itérations, temps CPU et norme finale du gradient en fonction de la dimension n pour les six algorithmes comparés.

La figure 9 révèle des comportements contrastés face à la difficulté croissante de Rosenbrock.

Itérations. Cauchy Point et Steihaug-CG atteignent systématiquement la limite de 10^3 itérations dès les grandes dimensions, signe d'une convergence insuffisante sur cette fonction très mal conditionnée. Dogleg, en revanche, converge en très peu d'itérations pour $n \leq 50$, mais atteint lui aussi le budget maximal à $n = 100$. Newton + Backtracking se distingue par un nombre d'itérations remarquablement stable (≈ 100) quelle que soit la dimension, témoignant de sa convergence quadratique locale. Les méthodes BFGS présentent un comportement intermédiaire, avec un nombre d'itérations croissant modérément.

Temps CPU. Malgré son faible nombre d'itérations, Newton + Backtracking reste compétitif en temps CPU jusqu'à $n = 100$, mais son coût par itération en $\mathcal{O}(n^3)$ le pénalise à $n = 1000$. À grande dimension, BFGS + Wolfe et BFGS + Armijo deviennent les méthodes les plus coûteuses en temps réel.

Norme finale du gradient. Le panneau de droite met en évidence un résultat préoccupant : la majorité des méthodes n'atteignent pas la tolérance $\varepsilon = 10^{-6}$ à grande dimension ($n \geq 500$), avec des normes finales de gradient comprises entre 10^0 et 10^2 . Seuls Dogleg et Steihaug-CG maintiennent une précision acceptable ($\|\nabla f\| \approx 10^{-8}$ à 10^{-9}) jusqu'à $n = 100$, avant de diverger à plus grande dimension faute de budget d'itérations suffisant.

5.5.3 Analyse détaillée en dimension $n = 10$

Afin d'examiner plus finement les dynamiques de convergence, trois méthodes représentatives sont comparées sur Rosenbrock 10D : Steihaug-CG (région de confiance), BFGS + Wolfe (quasi-Newton) et Newton + Backtracking (Newton exact).

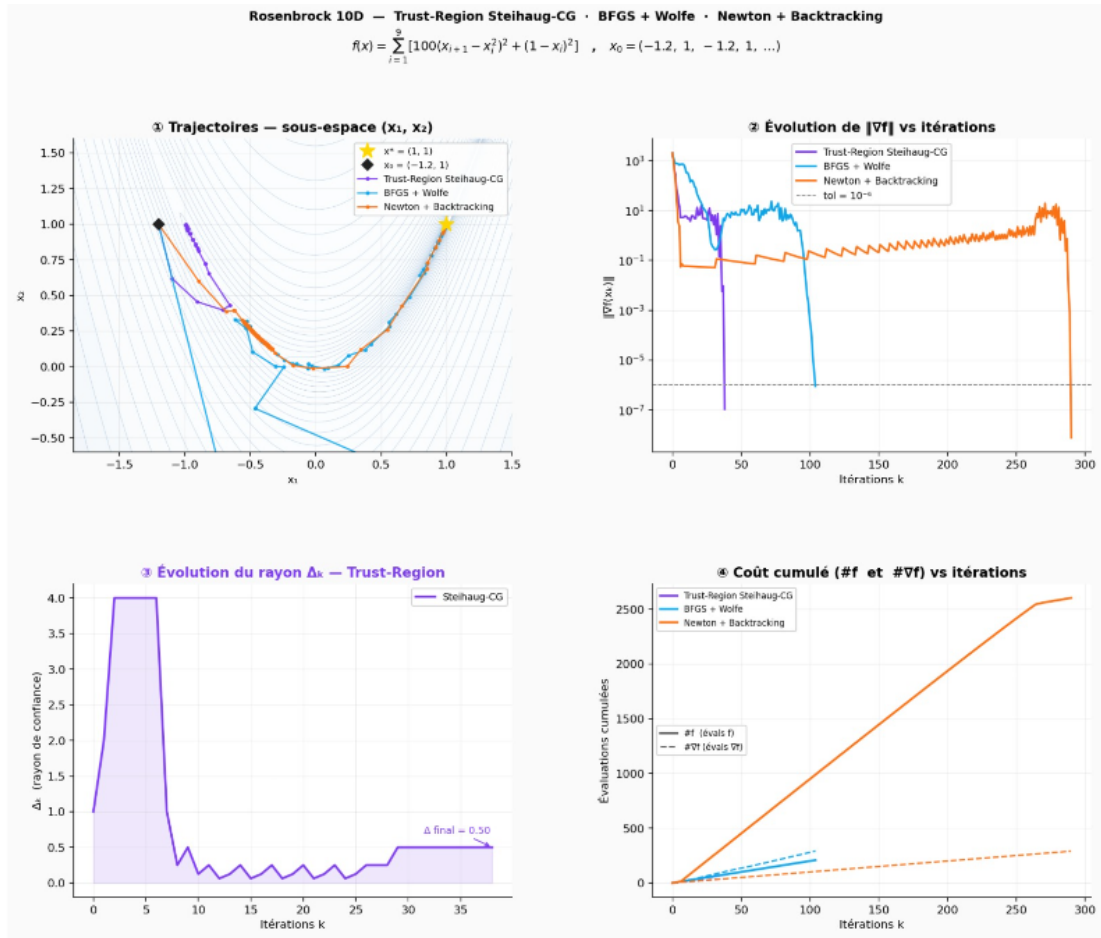


Figure 10: Analyse détaillée sur Rosenbrock $n = 10$, $x_0 = (-1.2, 1, -1.2, 1, \dots)^\top$. (a) Trajectoires projetées dans le sous-espace (x_1, x_2) ; (b) Évolution de $\|\nabla f(x_k)\|$ vs itérations ; (c) Évolution du rayon de confiance Δ_k (Steihaug-CG) ; (d) Coût cumulé en évaluations de f et ∇f .

Table 6: Résultats numériques sur Rosenbrock 10D ($n = 10$, $\varepsilon = 10^{-6}$).

Méthode	Itér.	$\ \nabla f\ _{\text{final}}$	$\#f$	$\#\nabla f$
Steihaug-CG	39	1.06×10^{-7}	77	39
BFGS + Wolfe	105	8.69×10^{-7}	209	292
Newton + BT	291	7.51×10^{-9}	2602	291

Trajectoires (panneau a). Les trois méthodes partent du même point $x_0 = (-1.2, 1)$ et convergent vers $x^* = (1, 1)$. Steihaug-CG emprunte un chemin direct et régulier le long de la vallée, tandis que BFGS + Wolfe effectue des oscillations transversales avant de s'aligner sur la vallée. Newton + Backtracking présente la trajectoire la plus erratique dans un premier temps, en raison des pas de Backtracking fréquents dans les premières itérations.

Convergence du gradient (panneau b). Steihaug-CG atteint la tolérance $\varepsilon = 10^{-6}$ en seulement 39 itérations avec une décroissance régulière et monotone de $\|\nabla f\|$. BFGS + Wolfe converge en 105 itérations avec quelques oscillations modérées. Newton + Backtracking présente une convergence lente et irrégulière sur les 200 premières itérations — le mal-conditionnement de la hessienne perturbe les pas

de Newton — avant d’atteindre une précision finale légèrement supérieure (7.51×10^{-9}).

Rayon de confiance (panneau c). Le rayon Δ_k de Steihaug-CG démarre à 1, augmente jusqu’à 4 lors des premières itérations favorables, puis décroît progressivement vers $\Delta_{\text{final}} \approx 0.50$ à mesure que l’algorithme approche du minimiseur et raffine ses pas.

Coût cumulé (panneau d). C’est sur ce critère que les différences sont les plus marquées. Steihaug-CG nécessite seulement 77 évaluations de f et 39 de ∇f , soit le coût le plus faible des trois. BFGS + Wolfe atteint un coût total modéré (≈ 500 évaluations cumulées). Newton + Backtracking accumule plus de 2600 évaluations de f en raison des nombreux rejets de pas par Backtracking, ce qui en fait la méthode la plus coûteuse sur ce problème malgré sa haute précision finale.

Conclusion. Sur la fonction de Rosenbrock, Steihaug-CG offre le meilleur compromis précision/coût à toutes les dimensions testées. Newton + Backtracking, bien que très précis en basse dimension, est pénalisé par son coût cubique et sa sensibilité au mal-conditionnement. Les méthodes BFGS constituent une alternative raisonnable mais moins efficace que les méthodes de région de confiance sur cette fonction difficile.

6. Étude théorique

6.1 Le Cauchy Point garantit une réduction suffisante

Proposition 6.1 (Réduction suffisante du Cauchy Point). *Le pas de Cauchy p_k^C satisfait :*

$$m_k(0) - m_k(p_k^C) \geq \frac{1}{2} \|g_k\| \min\left(\Delta_k, \frac{\|g_k\|}{\|B_k\|}\right). \quad (10)$$

Proof. $p_k^C = -\tau_k^* \frac{\Delta_k}{\|g_k\|} g_k$ avec $\tau_k^* = \min\left(1, \frac{\|g_k\|^3}{\Delta_k g_k^\top B_k g_k}\right)$ si $g_k^\top B_k g_k > 0$, sinon $\tau_k^* = 1$. La réduction le long de $-g_k$ pour un pas $\alpha > 0$ est $m_k(0) - m_k(-\alpha g_k) = \alpha \|g_k\|^2 - \frac{\alpha^2}{2} g_k^\top B_k g_k$.

Cas 1 ($g_k^\top B_k g_k \leq 0$) : $\alpha = \Delta_k / \|g_k\|$ donne $m_k(0) - m_k(p_k^C) = \Delta_k \|g_k\| \geq \frac{1}{2} \|g_k\| \Delta_k$.

Cas 2 ($g_k^\top B_k g_k > 0$) : le pas optimal non contraint est $\alpha^* = \|g_k\|^2 / (g_k^\top B_k g_k)$. Si $\alpha^* \leq \Delta_k / \|g_k\|$, la réduction vaut $\geq \frac{1}{2} \|g_k\|^2 / \|B_k\|$; sinon $\geq \frac{1}{2} \Delta_k \|g_k\|$. D’où (10). $\square$

6.2 Convergence globale

Théorème 6.2 (Convergence globale). *Si le pas p_k satisfait (10), alors $\liminf_{k \rightarrow \infty} \|\nabla f(x_k)\| = 0$.*

Preuve (esquisse). Si $\|\nabla f(x_k)\| \geq \epsilon > 0$, alors $\text{pred}_k \geq \frac{\epsilon}{2} \min(\Delta_k, \epsilon / \|B_k\|) > 0$. Pour Δ_k petit, $\rho_k \rightarrow 1$ (Taylor), le pas est accepté et $\text{ared}_k \geq \eta_1 \text{pred}_k > 0$. La somme $\sum_k \text{ared}_k$ diverge, contradiction avec la bornitude de f . $\square$

6.3 Taux de convergence asymptotique

- **Cauchy Point** : convergence linéaire au taux $1 - 1/\kappa(B_k)$, proche de 1 si $\kappa \gg 1$ (10,7 % en Cat. A).
- **Dogleg / Steihaug-CG** : convergence superlinéaire si $B_k = \nabla^2 f(x_k)$ et $x_k \rightarrow x^*$.

- **BFGS** : convergence superlinéaire sous les conditions de Wolfe.
- **Newton-BT** : convergence quadratique locale, $\|x_{k+1} - x^*\| = \mathcal{O}(\|x_k - x^*\|^2)$.

Cohérence empirique : Newton-BT converge en 9 itérations médianes, Dogleg en 11, Steihaug-CG en 21, Cauchy Point stagne à $K_{\max} = 1000$.

6.4 Cas pathologiques

Situations où trust-region échoue

- **Dogleg / Hessienne indéfinie** : $B_k \succ 0$ requis. Cat. C : 50 % seulement.
- **Cauchy Point / mal conditionné** : 10,7 % en Cat. A, pas zig-zagants dans les vallons.
- **Steihaug-CG / grande dimension** : coût CG $\mathcal{O}(n^2)$, temps CPU significatif à $n = 1000$.

Situations où la recherche linéaire échoue

- **BFGS-Wolfe / Hessienne indéfinie** : 26 exceptions, 26 % en Cat. C.
- **Newton-BT / Hessienne singulière** : direction instable, bascule sur $-g_k$ dégrade la convergence.
- **BFGS / mal conditionné** : 62,7 % en Cat. A, 111 itérations vs 6 pour Dogleg.

7. Discussion

7.1 Interprétation des résultats

Q1 — Trust-region sur problèmes mal conditionnés. Oui, significativement : 82,7 % (Dogleg/Steihaug-CG) vs 62–68 % (BFGS). La région de confiance s’adapte via Δ_k et évite la stagnation en vallée que subit la recherche linéaire sur des problèmes de conditionnement $\kappa = 10^5$.

Q2 — Approche évitant mieux les minima locaux. Steihaug-CG (89,1 % en Cat. B) grâce à la détection de courbure négative. Sur les fonctions fortement multi-modales (Rastrigin, Ackley), les deux familles sont comparables : aucune méthode de second ordre n’est conçue pour l’optimisation globale.

Q3 — Coût du sous-problème trust-region. Justifié pour Dogleg et Steihaug-CG : 11 itérations médianes (Dogleg) vs 82,5 (BFGS-Armijo). Le Cauchy Point ne vaut pas son coût en Cat. A (4002 évaluations vs 27 pour Dogleg).

Q4 — Méthode la plus robuste. Newton-BT : 78,9 % global, zéro exception, 76 % en Cat. C. Limite : coût $\mathcal{O}(n^3)$ impraticable à grande dimension.

Q5 — Évolution avec la dimension. À $n = 1000$, Newton-BT devient le plus lent en CPU malgré le moins d’itérations. Steihaug-CG est la seule méthode scalable de $n = 10$ à $n = 1000$.

7.2 Recommandations pratiques

- **Mal conditionné, $n \leq 100$:** **Dogleg** (82,7 %, 6 it., CPU 0,045 s). Si $B_k \neq 0$: Steihaug-CG.
- **Mal conditionné ou non-convexe, $n \geq 500$:** **Steihaug-CG** (82,7 % Cat. A, 89,1 % Cat. B).
- **Hessienne indéfinie :** **Newton-BT** ($n \leq 100$) ou **Steihaug-CG** (n grand).
- **Structure inconnue :** **Newton-BT** (78,9 %) sauf si n grand. Éviter Cauchy Point et BFGS-Wolfe.

7.3 Limitations de l’étude

Représentativité. Les problèmes creux (Hessiennes sparse) ne sont pas représentés alors qu’ils dominent les applications industrielles.

Exécution sérielle. Durée ≈ 5 h sur une seule machine.

Absence de régularisation. Un Cauchy Point avec préconditionnement pourrait significativement améliorer les résultats en Cat. A.

Hessienne exacte. Une comparaison avec L-BFGS serait plus réaliste à grande dimension.

8. Conclusion

Cette étude comparative sur 180 instances aboutit aux conclusions suivantes.

Cat. A (mal conditionné) : Dogleg et Steihaug-CG dominant (82,7 % vs 62–68 % pour BFGS). La gestion simultanée direction/amplitude via Δ_k est décisive.

Cat. B (non-convexe) : Steihaug-CG atteint 89,1 %, meilleur score absolu, grâce à la détection de courbure négative.

Cat. C (Hessienne indéfinie) : Newton-BT domine (76 %), BFGS-Wolfe s'effondre (26 %). La famille algorithmique est moins discriminante que le choix spécifique de méthode.

Scalabilité : Steihaug-CG est la seule méthode maintenant un compromis itérations/CPU raisonnable de $n = 10$ à $n = 1000$. Newton-BT ($\mathcal{O}(n^3)$) est impraticable au-delà de $n = 500$.

Synthèse : trust-region 62,6 %, recherche linéaire/Newton 65,7 % en moyenne. Les écarts par catégorie sont importants — le choix doit être guidé par la structure du problème, pas par une préférence générale.

En perspective : extension aux méthodes L-BFGS, aux problèmes creux, et exploration de stratégies hybrides (trust-region avec Hessienne quasi-Newton) pour combiner les avantages des deux familles.